



life.augmented

Add the secure firmware update service

TOMAS Team

Add the secure firmware update service

6

Add the secure firmware update service

Agenda Details

- Timing : 45 minutes
- Purpose
 - Show a key feature of the SM : secure update capability
- Process
 - CubeMX : add the secure manager firmware update service and generate code
 - CubeIDE add ymodem loader and use the fw update psa api
 - Practice the update
 - Explain the STM32Trustee-sm secure firmware update service principle with some details
- Payoff
 - PSA API somehow quite simple to integrate and to understand
 - Secure boot and secure firmware update made accessible with maximum security and minimum effort!

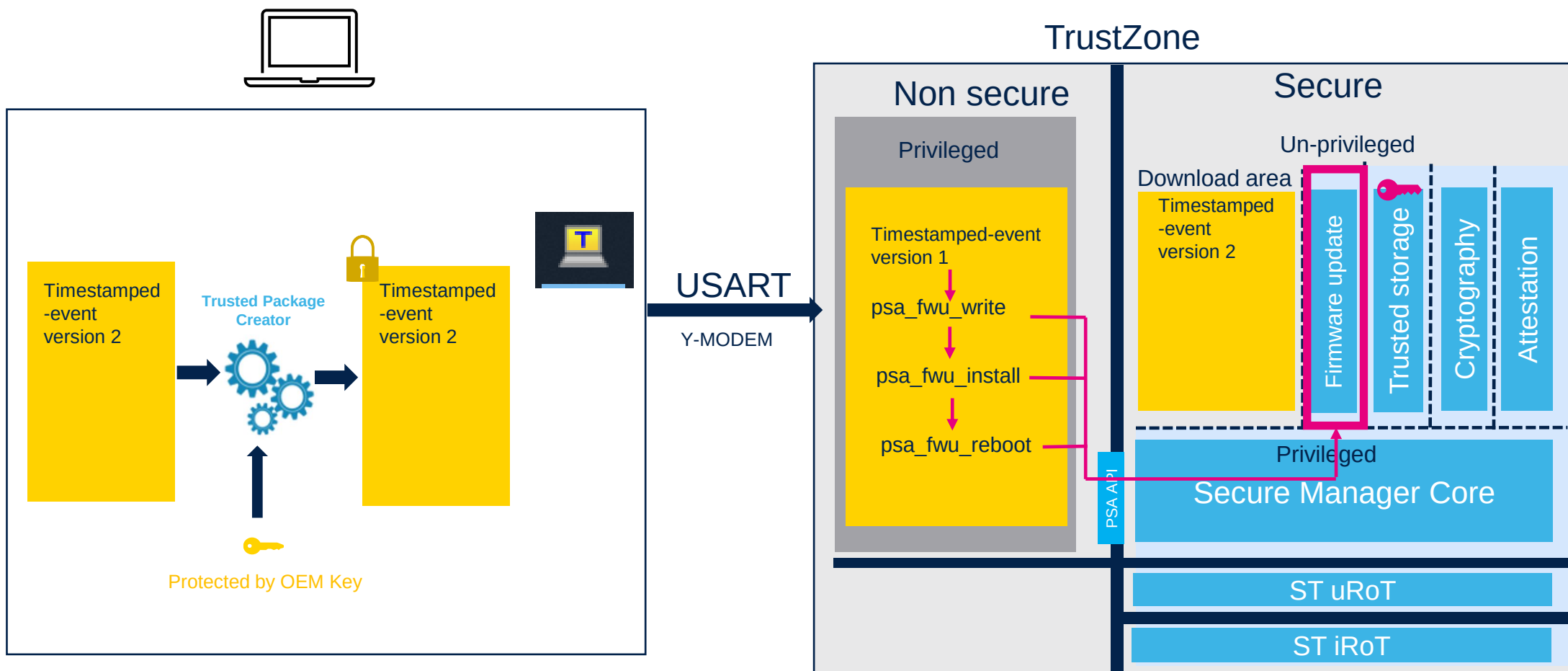


Our application is now ready but...

- Our secure TimeStamped_Event_Detection is now ready but now we would like to have the capability to update it securely ...
- One of the secure service offered by secure manager is firmware update, so let's add this capability.
- How to address this ?
 - We need to add the capability in our application to download a new version of the firmware (Ymodem protocol over the usart)
 - We need to add the call to the Firmware Update PSA API :
 - psa_fwu_write : write the new firmware in the download slot
 - psa_fwu_install : request the installation of a new firmware
 - psa_fwu_request_reboot : trig a reboot to start installation
 - psa_fwu_accept : at first execution validate the current application



Secure firmware update algorithm !

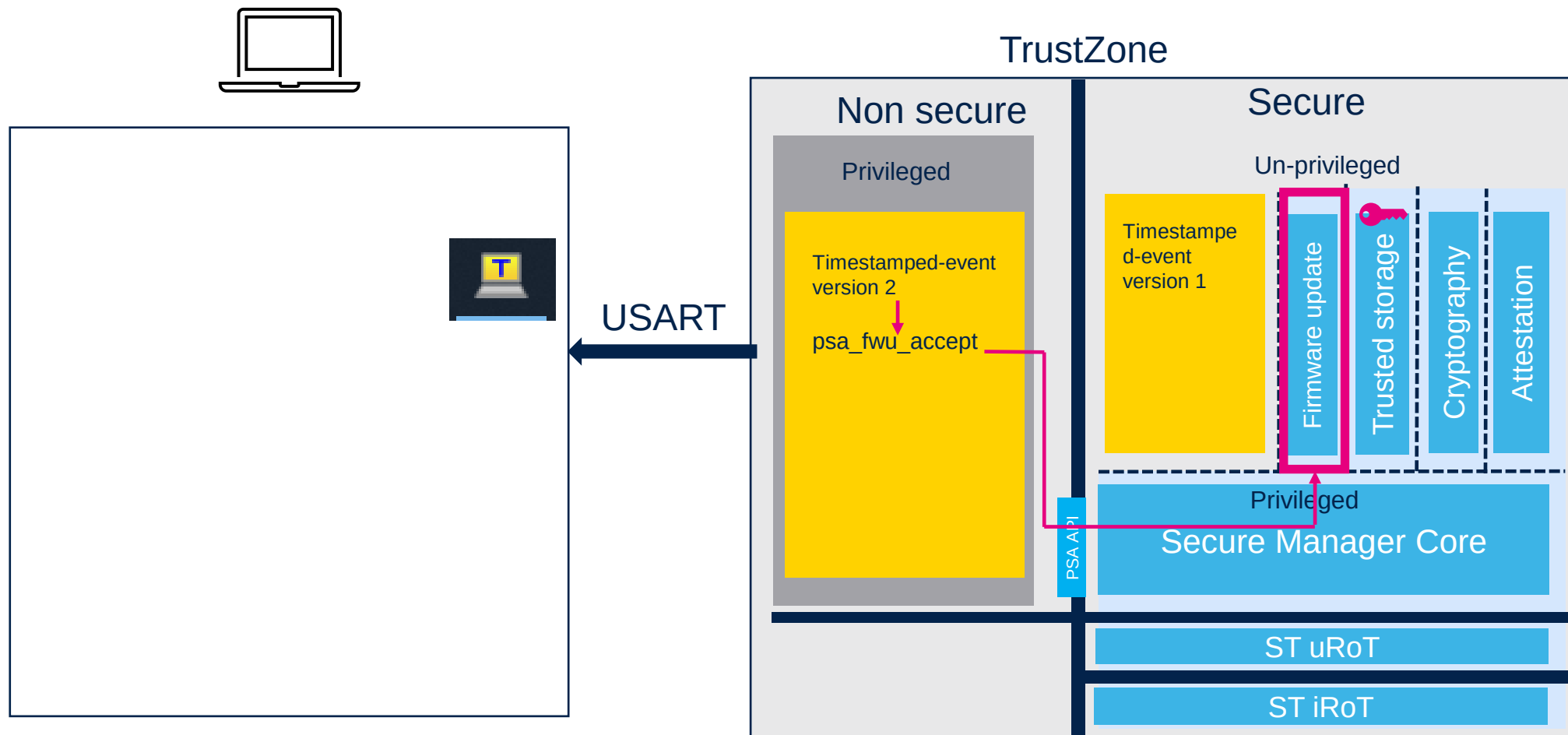


Firmware update in secure boot

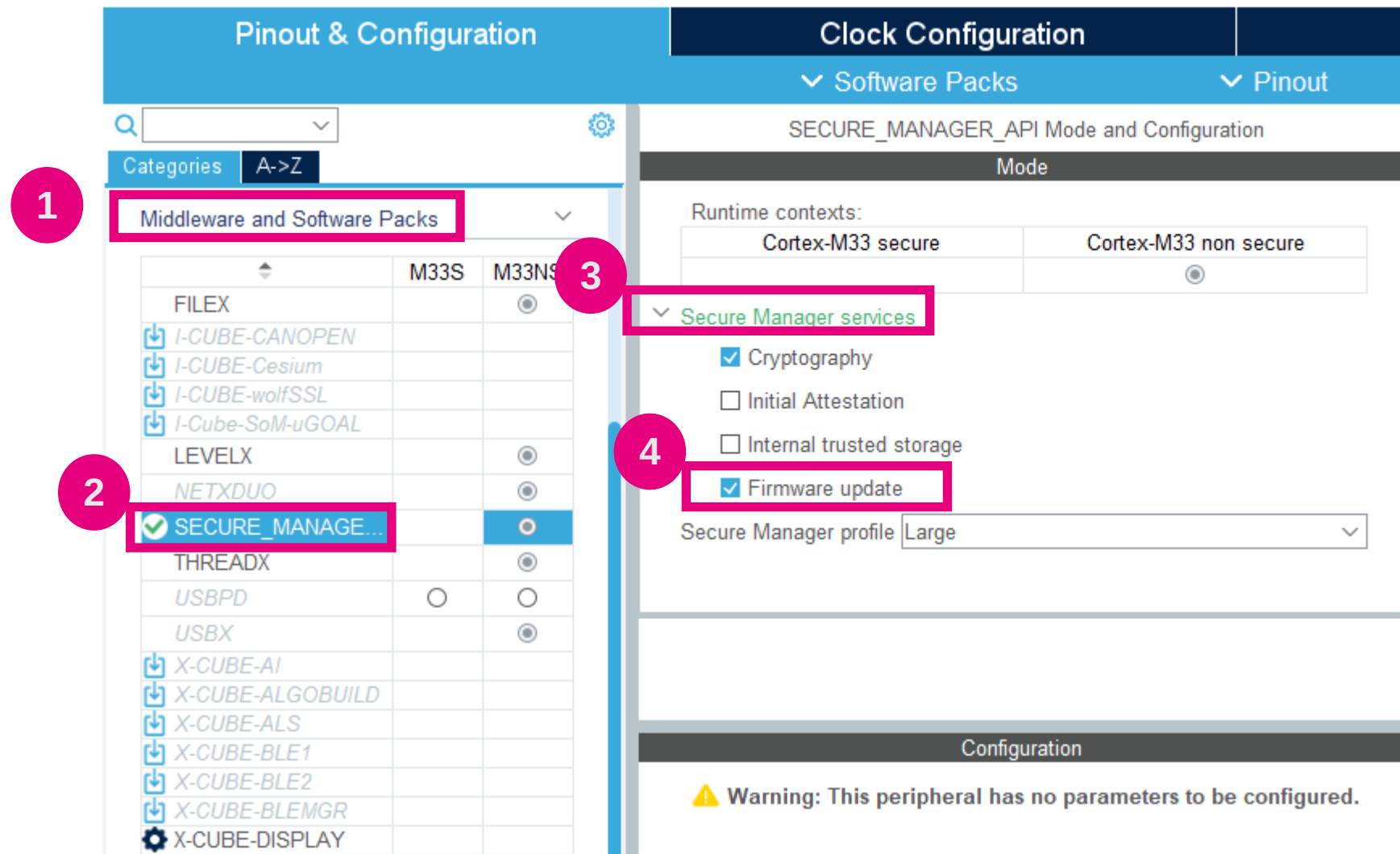
- After Reset, STuROT will detect a new firmware in download area.
- It will check authenticity of the firmware
- Then it will exchange (swap) the content of download area and active area
- This exchange is done flash sector by sector using a scratch intermediate area
- For more information about this swap mechanism please check MCUBoot documentation



Secure firmware validation!



Add the Firmware update API !



The screenshot shows the STM32CubeMX Pinout & Configuration window. The 'Pinout & Configuration' tab is active. The 'Software Packs' section is expanded, and the 'Middleware and Software Packs' category is selected. The 'SECURE_MANAGER_API' package is highlighted in the list. The 'Secure Manager services' section is expanded, and the 'Firmware update' checkbox is checked. The 'Secure Manager profile' is set to 'Large'. A warning message at the bottom states: 'Warning: This peripheral has no parameters to be configured.'

1. Categories: A-Z

2. Middleware and Software Packs

3. SECURE_MANAGER_API

4. Secure Manager services

5. Firmware update

6. Secure Manager profile: Large

Warning: This peripheral has no parameters to be configured.



Activate the CRC and generate the code

Pinout & Configuration

Categories: A->Z

- System Core
- Analog
- Timers
- Connectivity
- Multimedia
- Security
- Computing
- Middleware and Software Packs
- Trace and Debug
- Power and Thermal

	M33S	M33NS
CORDIC	☐	☐
☒ CRC	☐	☐
FMAC	☐	☐

Clock Configuration

Software Packs

Pinout

CRC Mode and Configuration

Mode

Runtime contexts:

- Cortex-M33 secure ☐
- Cortex-M33 non secure ☒

☒ Activated

Configuration

Reset Configuration

Parameter Settings User Constants

Configure the below parameters :

Search (Ctrl+F)

Basic Parameters

- Default Polynomial State: Enable
- Default Init Value State: Enable

Advanced Parameters

- Input Data Inversion Mode: None
- Output Data Inversion Mode: Disable
- Input Data Format: Bytes

Ymodem protocol used to transfer update file needs CRC

MX Code Generation

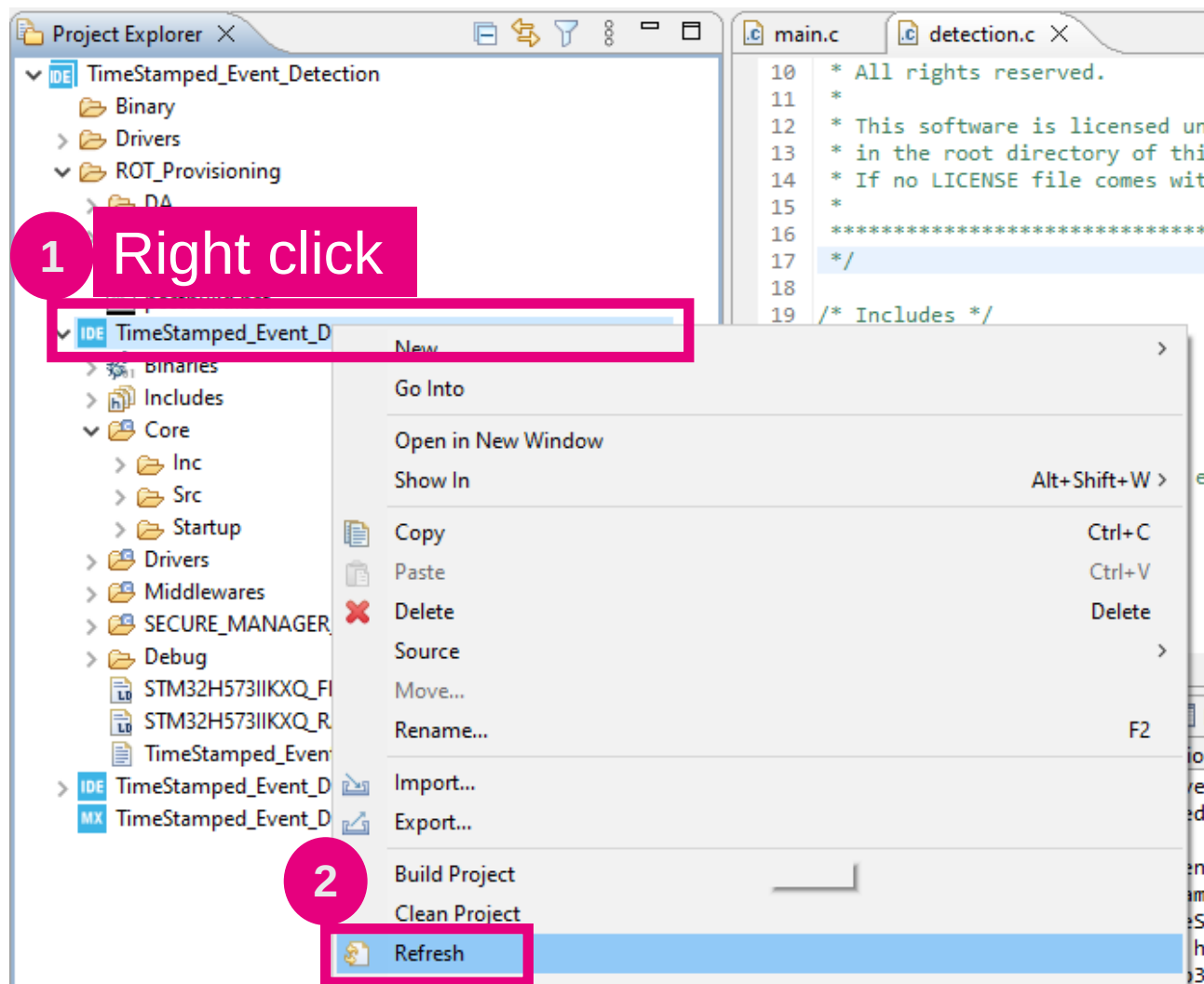
The Code is successfully generated under :

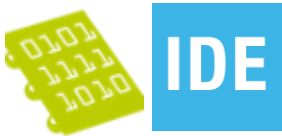
C:/Training/STM32H5_Security_WS/Hands-on/TimeStamped_Event_Detection

Project language : C

Open Folder Open Project Close

Refresh the project !





Add source file to handle the firmware update

1 Drag and drop: update_fw.c and ymodem.c
In Core\Src folder

2 Select how files should be imported into the project:

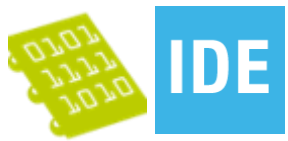
- ☒ Copy files
- ☐ Link to files

☒ Create link locations relative to: PROJECT_LOC

[Configure Drag and Drop Settings](#)

3 OK Cancel

Material location :
C:\Training\STM32H5_Security_WS\Hands-on-material\
TimeStamped_Event_Detection\Project_patch_files\Src



Add header files to handle the firmware update

1

Drag and drop: update_fw.h and ymodem.h
In Core\Inc folder

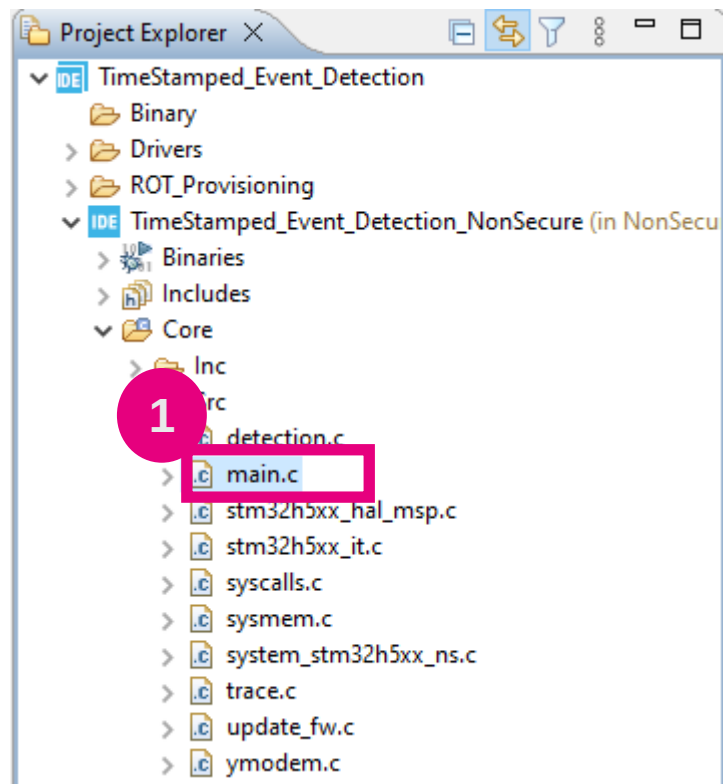
The screenshot shows the IDE interface with the Project Explorer on the left and the File Operation dialog in the center. The Project Explorer shows the project structure with the 'Core' folder expanded, and the 'Inc' subfolder selected. The File Operation dialog is open, showing the 'Copy files' option selected. The 'update_fw.h' and 'ymodem.h' files are highlighted in the 'Name' list. The 'OK' button is highlighted in the dialog.

2

3

Material location :
C:\Training\STM32H5_Security_WS\Hands-on-material\
TimeStamped_Event_Detection\Project_patch_files\Inc

Modify the code



2

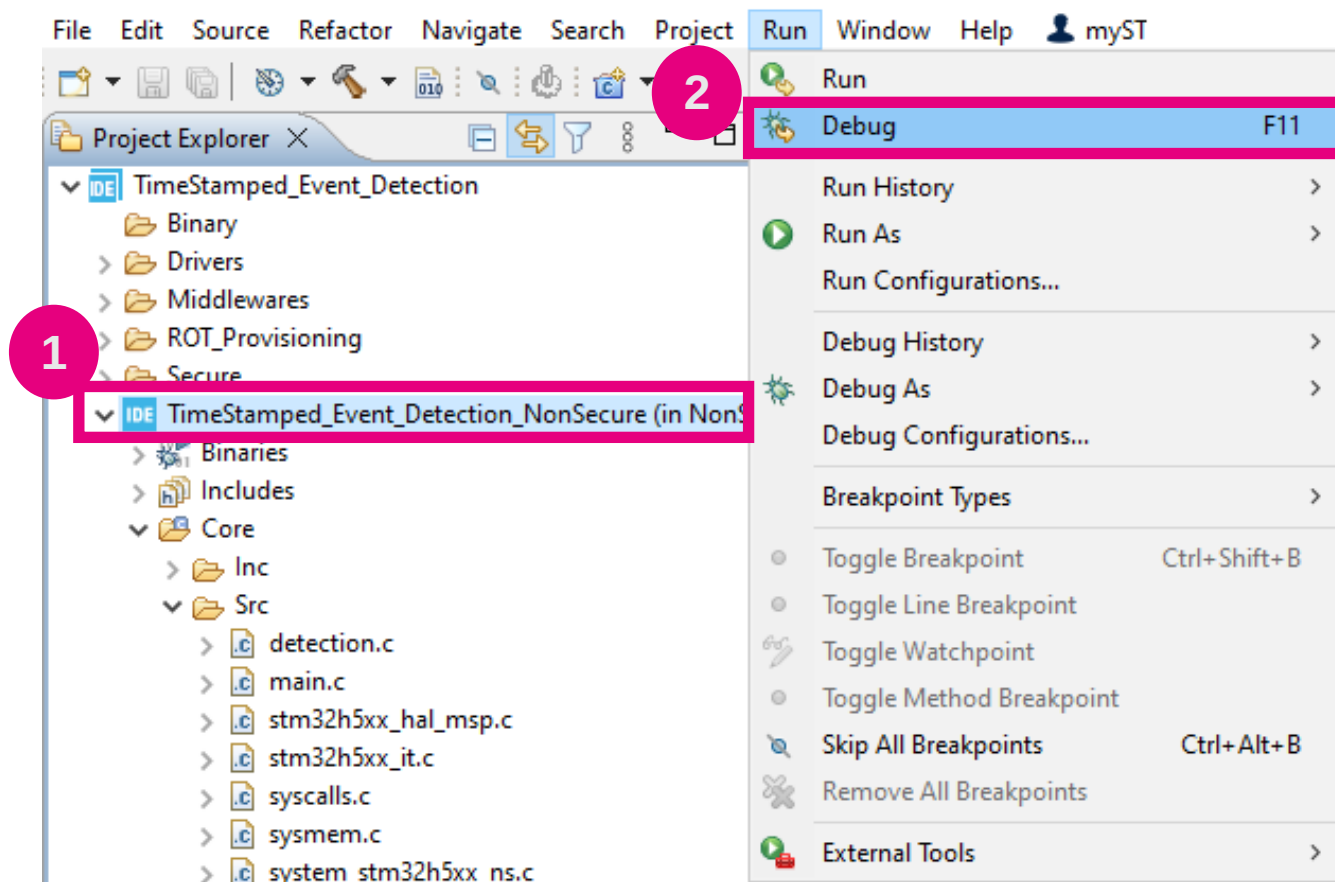
```
24 /* USER CODE BEGIN Includes */
25 #include "detection.h"
26 #include "update_fw.h"
27 /* USER CODE END Includes */
```

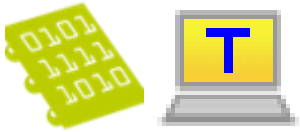
3

```
115 while (1)
116 {
117     ProcessEventDetection();
118     CheckUpdateRequest();
119     /* USER CODE END WHILE */
120
121     /* USER CODE BEGIN 3 */
122 }
```

Material location :
 C:\Training\STM32H5_Security_WS\Hands-on-material\
 TimeStamped_Event_Detection\CodeToAdd.txt

Compile and debug



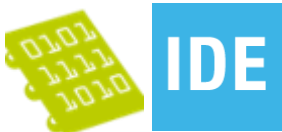


Firmware update is ready to be tested !

```
VT COM77 - Tera Term VT
File Edit Setup Control Window Help
*****
Timestamped event detection version 1
*****
Active slot state after boot : PSA_IMAGE_UNDEFINED
Download slot state after boot : PSA_IMAGE_UNDEFINED
Please press key '1' to trigger a firmware update
█

VT COM77 - Tera Term VT
File Edit Setup Control Window Help
*****
Timestamped event detection version 1
*****
Active slot state after boot : PSA_IMAGE_UNDEFINED
Download slot state after boot : PSA_IMAGE_UNDEFINED
Please press key '1' to trigger a firmware update

*****Firmware update triggered *****
Active slot state before download : PSA_IMAGE_UNDEFINED
Download slot state before download : PSA_IMAGE_UNDEFINED
Please send binary application using File> Transfer> YMODEM> Send
...█
```



Generate a version 2 of the application

The screenshot shows an IDE interface with the following components:

- Project Explorer:** Displays a project structure for 'TimeStamped_Event_Detection'. The 'NonSecure' configuration is selected, showing files like 'env.bat', 'postbuild.bat', and 'TimeStamped_Event_Detection_NonSecure.launch'.
- main.c:** The source code is open, showing initialization functions and a while loop. Lines 109-110 are highlighted with a pink box:

```
109 printf("Timestamped event detection version 2\r\n");  
110 printf("*****\r\n");
```
- Console:** Shows the build output for 'TimeStamped_Event_Detection_NonSecure'. The output includes the build command, compilation steps, and a final success message. The success message is highlighted with a pink box:

```
"C:/Training/STM32H5_Security_WS/Hands-on/TimeStamped_Event_Detection/NonSecure/./ROT_Provisioning/postbuild.bat" "nonsecure"  
TPC success
```



What is done in the post-build script ?

TimeStamped_Event_Detection_NonSecure.hex



Trusted Package
Creator



SM_Authentication.pem



SM_Encryption.pem



TimeStamped_Event_Detection_NonSecure_enc_sign.bin

TimeStamped_Event_Detection_NonSecure_enc_sign.hex

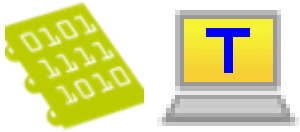


Where those keys come from?

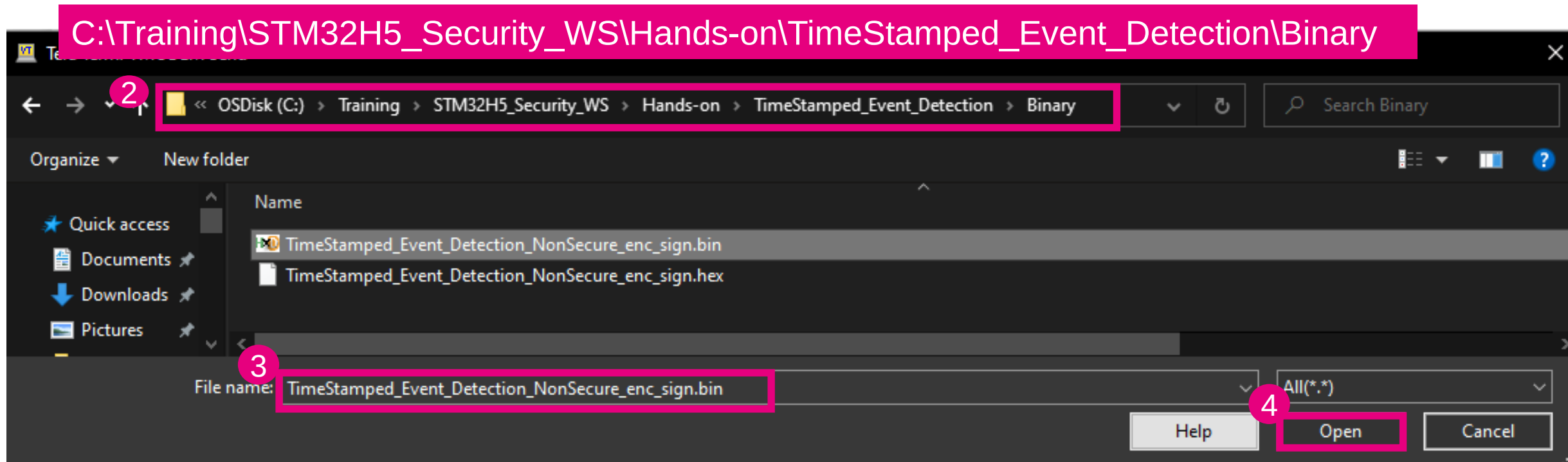
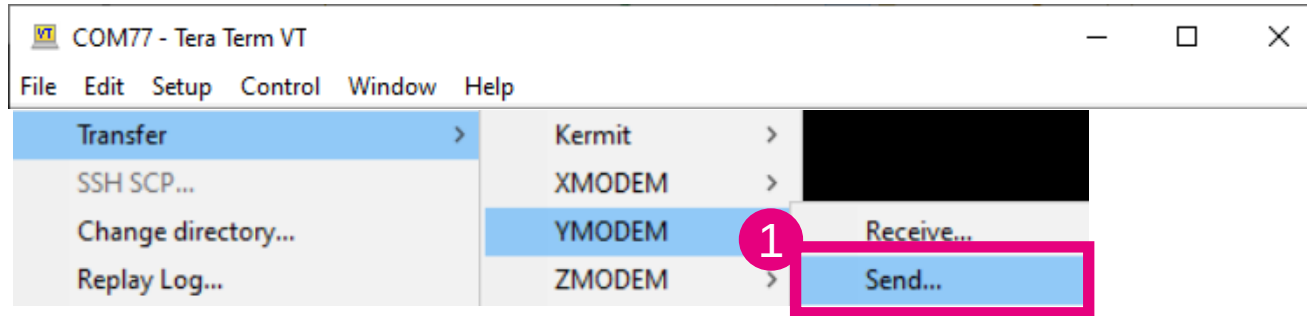
- Crypto scheme : elliptic curves (ECDSA-p256 / ECIES-p256)
- Keys has been provisioned when you install the secure manager on your target
- Information from the log :

```
* OEM Keys configuration:
  From TrustedPackageCreator (tab H5-OBkey)
  Select SM_Config_Keys.xml (in \ROT_Provisioning\SM\Config)
  Warning: Default keys must NOT be used in a product. Make sure to regenerate your own keys
  Update the configuration (if/as needed) then generate OBkey file
  Press any key to continue...
  Processing OBKey file...
  Successful SM_Config_Keys.obk file generation
```

TimeStamped_Event_Detection\ROT_Provisioning\SM\Config\SM_Config_Keys.xml



Download the new version...





Firmware update

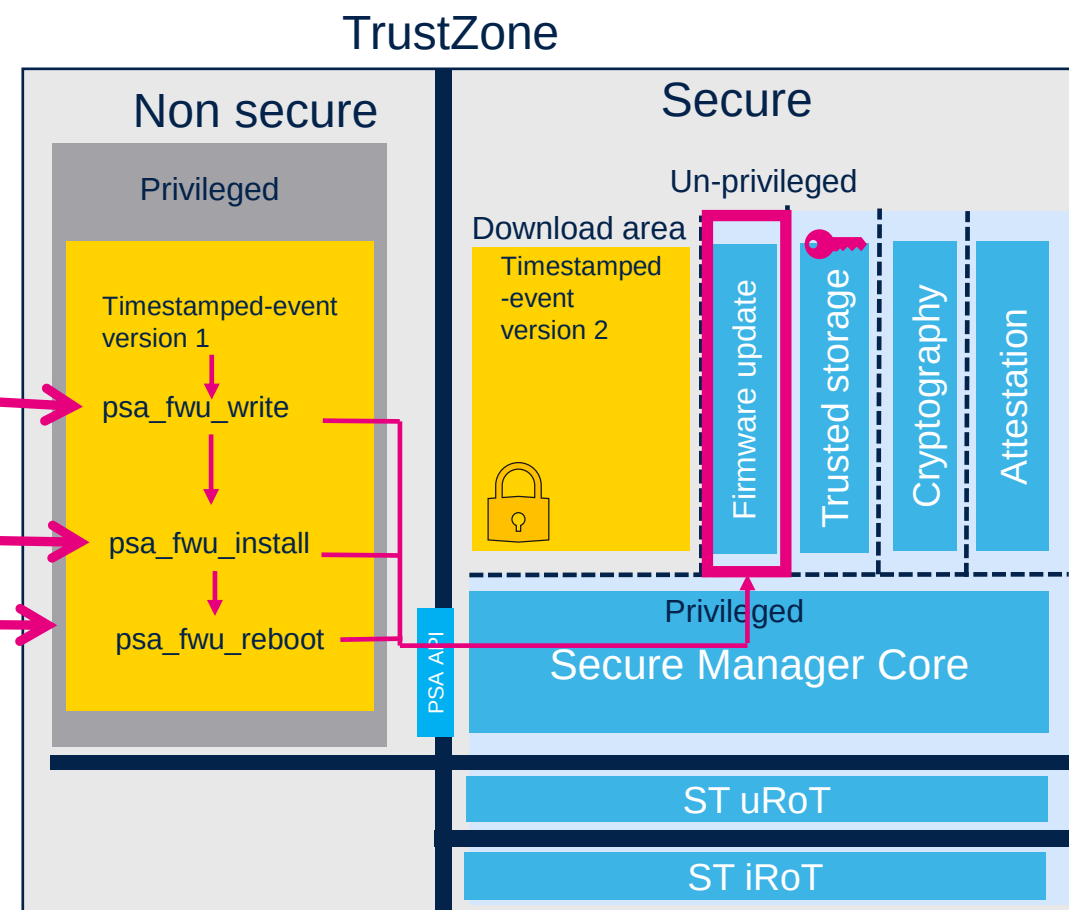
```
VT COM77 - Tera Term VT
File Edit Setup Control Window Help
*****
Timestamped event detection version 1
*****
Active slot state after boot : PSA_IMAGE_UNDEFINED
Download slot state after boot : PSA_IMAGE_UNDEFINED
Please press key '1' to trigger a firmware update

*****Firmware update triggered *****
Active slot state before download : PSA_IMAGE_UNDEFINED
Download slot state before download : PSA_IMAGE_UNDEFINED
Please send binary application using File> Transfer> YMODEM> Send
...e_result = 0 , 3

Firmware download successful
Received bytes: 51858

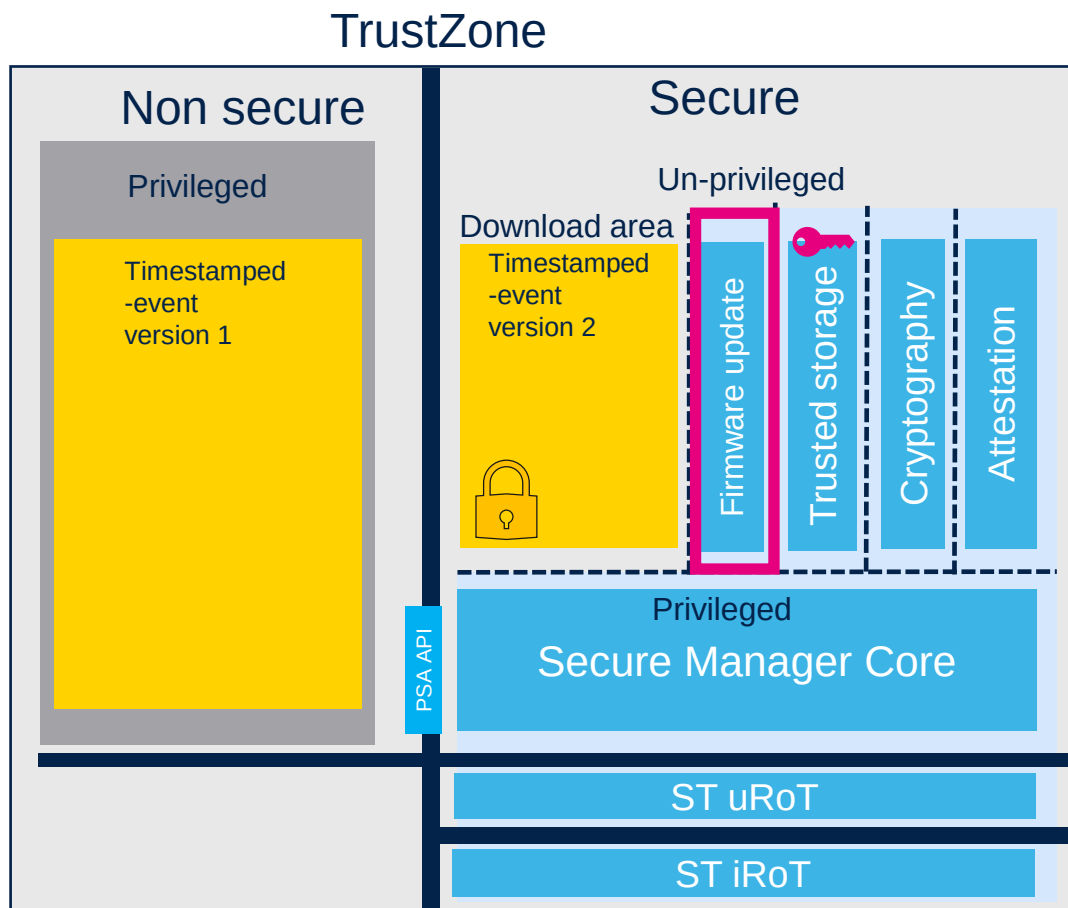
Active slot state after download : PSA_IMAGE_UNDEFINED
Download slot state after download : PSA_IMAGE_CANDIDATE
Request firmware installation
Install successful
Active slot state after install request : PSA_IMAGE_UNDEFINED
Download slot state after install request : PSA_IMAGE_REBOOT_NEEDED
Request reboot ...

*****
Timestamped event detection version 2
*****
Active slot state after boot : PSA_IMAGE_INSTALLED
Download slot state after boot : PSA_IMAGE_UNDEFINED
Please press key '1' to trigger a firmware update
```





What is done when reboot ?



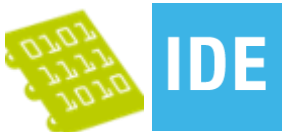
- At startup STuROT check the download slot
- Firmware found, it will decrypt it and check the signature
- If ok, it will swap the active and download slot
- Then platform boots normally



Why image validation is not needed here ?

```
VT COM77 - Tera Term VT
File Edit Setup Control Window Help
*****
Timestamped event detection version 1
*****
Active slot state after boot : PSA_IMAGE_UNDEFINED
Download slot state after boot : PSA_IMAGE_UNDEFINED
Please press key '1' to trigger a firmware update
*****Firmware update triggered*****
Active slot state before download : PSA_IMAGE_UNDEFINED
Download slot state before download : PSA_IMAGE_UNDEFINED
Please send binary application using File> Transfer> YMODEM> Send
...e_result = 0 , 3
Firmware download successful
Received bytes: 51858
Active slot state after download : PSA_IMAGE_UNDEFINED
Download slot state after download : PSA_IMAGE_CANDIDATE
Request firmware installation
Install successful
Active slot state after install request : PSA_IMAGE_UNDEFINED
Download slot state after install request : PSA_IMAGE_REBOOT_NEEDED
Request reboot ...
*****
Timestamped event detection version 2
*****
Active slot state after boot : PSA_IMAGE_INSTALLED
Download slot state after boot : PSA_IMAGE_UNDEFINED
Please press key '1' to trigger a firmware update
█
```

- PSA_IMAGE_UNDEFINED means non secure image is not signed. Reason : downloaded by debugger
- In this case, no need to validate the new image because no rollback could be possible



Generate a version 3 of the application

The screenshot shows an IDE interface with the following components:

- Project Explorer:** Displays a project structure for 'TimeStamped_Event_Detection'. The 'Core' folder is expanded, showing 'Inc', 'Src', and 'Startup' subfolders. The 'Src' folder is selected, showing 'main.c'.
- main.c:** The code editor shows the following code:

```
99  /* Initialize all configured peripherals */
100 MX_GPIO_Init();
101 MX_MEMORYMAP_Init();
102 MX_ICACHE_Init();
103 MX_RTC_Init();
104 MX_USART1_UART_Init();
105 MX_SECURE_MANAGER_API_Init();
106 MX_CRC_Init();
107
108 printf("*****\r\n");
109 printf("Timestamped event detection version 3\r\n");
110 printf("*****\r\n");
111 /* USER CODE END 2 */
112
113 /* Infinite loop */
114 /* USER CODE BEGIN WHILE */
115 while (1)
116 {
117     ProcessEventDetection();
```
- CDT Build Console:** The console output shows the build process for 'TimeStamped_Event_Detection_NonSecure'. The output includes the following text:

```
18:24:17 **** Incremental Build of configuration Debug for project TimeStamped_Event_Detection_NonSecure ****
make -j8 all
arm-none-eabi-gcc "../Core/Src/main.c" -mcpu=cortex-m33 -std=gnu11 -g3 -DDEBUG -DUSE_HAL_DRIVER -DSTM32H573xx -DTFM_PSA_API -DSECURE_MANAGER_PROFILE_LF
arm-none-eabi-gcc -o "TimeStamped_Event_Detection_NonSecure.elf" @"objects.list" -mcpu=cortex-m33 -T"C:\Training\STM32H5_Security_WS\Hands-on\TimeSta
Finished building target: TimeStamped_Event_Detection_NonSecure.elf

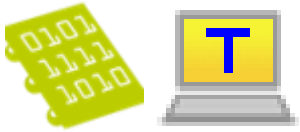
arm-none-eabi-size TimeStamped_Event_Detection_NonSecure.elf
arm-none-eabi-objdump -h -S TimeStamped_Event_Detection_NonSecure.elf > "TimeStamped_Event_Detection_NonSecure.list"
arm-none-eabi-objcopy -O ihex TimeStamped_Event_Detection_NonSecure.elf "TimeStamped_Event_Detection_NonSecure.hex"
text data bss dec hex filename
50120 276 3564 53960 d2c8 TimeStamped_Event_Detection_NonSecure.elf
Finished building: default.size.stdout

Finished building: TimeStamped_Event_Detection_NonSecure.hex

Finished building: TimeStamped_Event_Detection_NonSecure.list

"C:/Training/STM32H5_Security_WS/Hands-on/TimeStamped_Event_Detection/NonSecure/./ROT_Provisioning/postbuild.bat" "nonsecure"
TPC success

18:24:27 Build Finished. 0 errors, 0 warnings. (took 10s.46ms)
```



Download the new version...

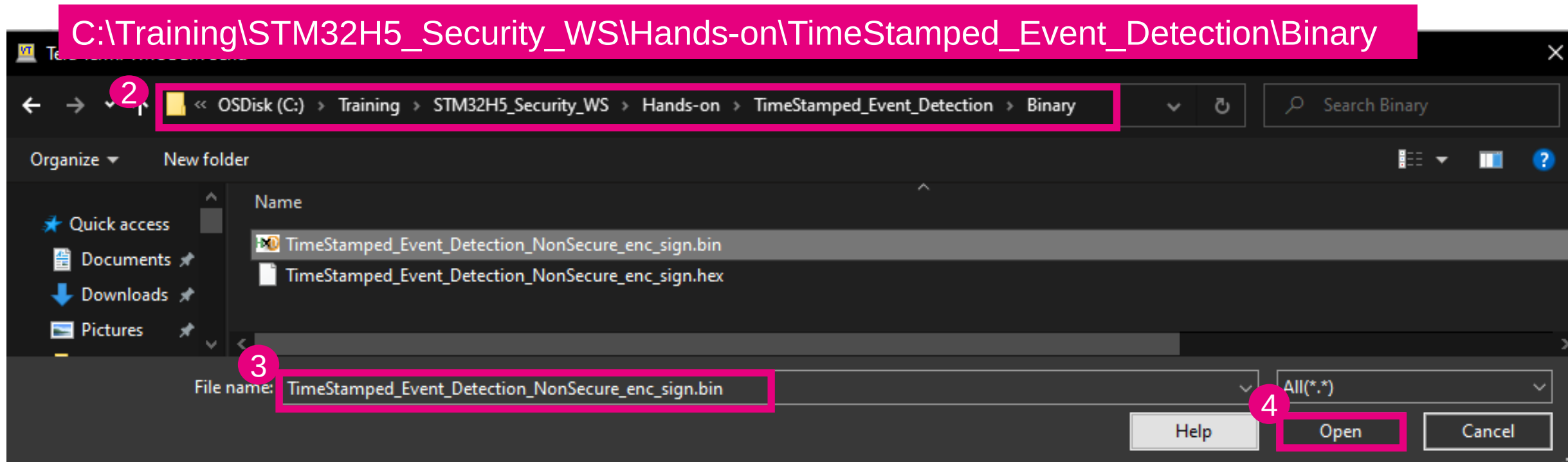
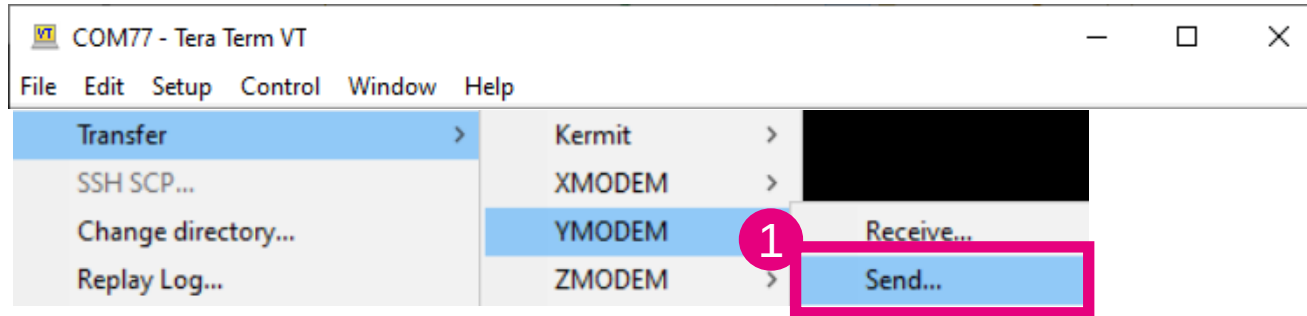




Image validation mechanism

File Edit Setup Control Window Help

```
*****
Timestamped event detection version 2
```

```
Active slot state after boot : PSA_IMAGE_INSTALLED
Download slot state after boot : PSA_IMAGE_UNDEFINED
```

```
Please press key '1' to trigger a firmware update
```

```
*****Firmware update triggered*****
```

```
Active slot state before download : PSA_IMAGE_INSTALLED
Download slot state before download : PSA_IMAGE_UNDEFINED
Please send binary application using File> Transfer> YMODEM> Send
.e_result = 0 , 3
```

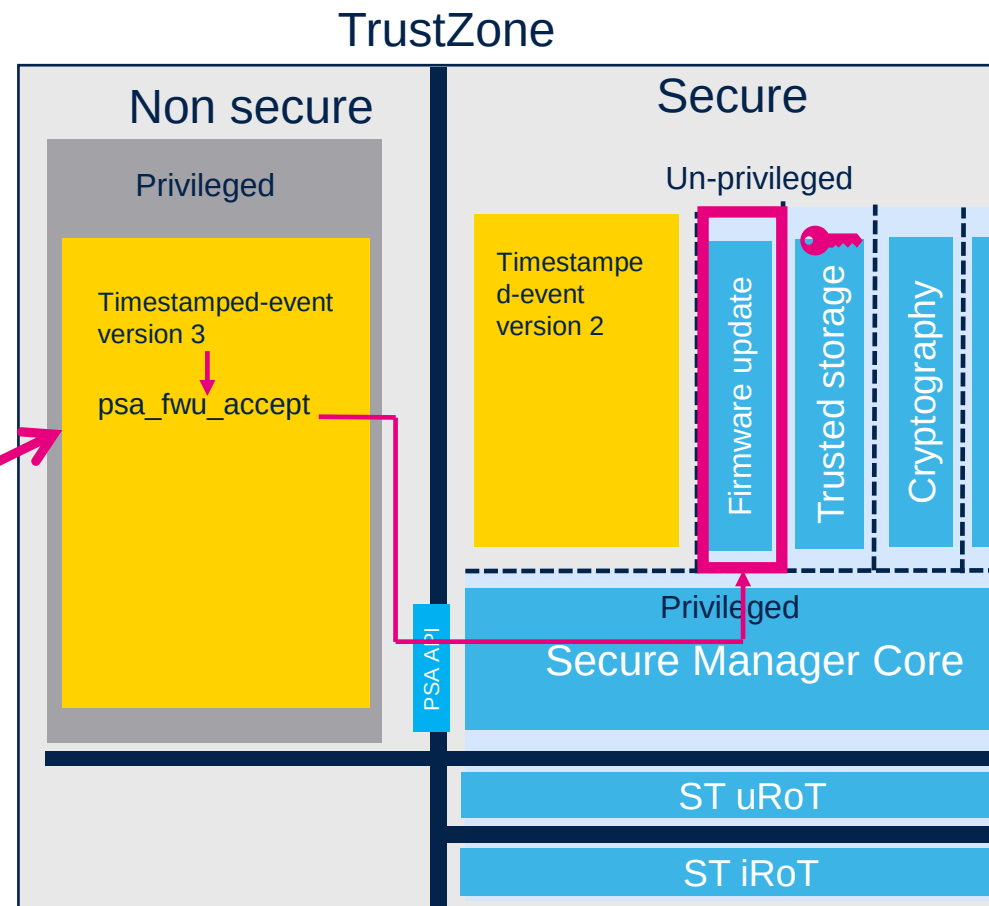
```
Firmware download successful
Received bytes: 52210
```

```
Active slot state after download : PSA_IMAGE_INSTALLED
Download slot state after download : PSA_IMAGE_CANDIDATE
Request firmware installation
Install successful
Active slot state after install request : PSA_IMAGE_INSTALLED
Download slot state after install request : PSA_IMAGE_REBOOT_NEEDED
Request reboot ...
```

```
*****
Timestamped event detection version 3
```

```
Active slot state after boot : PSA_IMAGE_PENDING_INSTALL
Download slot state after boot : PSA_IMAGE_PENDING_INSTALL
Active slot validated : PSA_IMAGE_INSTALLED
```

```
*****
Please press key '1' to trigger a firmware update
```





Implementation details: update_fw.c

```
char psa_image_state[NUMBER_OF_STRING][MAX_STRING_SIZE] =
{
    "PSA_IMAGE_UNDEFINED",
    "PSA_IMAGE_CANDIDATE",
    "PSA_IMAGE_INSTALLED",
    "PSA_IMAGE_REJECTED",
    "PSA_IMAGE_PENDING_INSTALL",
    "PSA_IMAGE_REBOOT_NEEDED"
};
```

https://wiki.st.com/stm32mcu/wiki/Security:SMAK_for_STM32H5#PSA_firmware_update_service

State	Value	Description
UNDEFINED	0x0	Empty download slot. Or download slot containing a former image whereas new image on the active slot has been successfully installed.
CANDIDATE	0x1	Image candidate present in the download slot.
INSTALLED	0x2	Image accepted by the nonsecure application (update agent) in the active slot.
REJECTED	0x3	Image not accepted by the nonsecure application (update agent), swapped back to download slot.
PENDING_INSTALL	0x4	Image candidate in the active slot, waiting for image acceptance from the nonsecure application (update agent).
REBOOT_NEEDED	0x5	Reboot is needed to finalize the image candidate swap from download slot to active slot.



Implementation details: update_fw.c

```
static void DownloadAndInstallNewFirmware(void)
{
    psa_image_id_t image_id;
    psa_status_t res;
    uint32_t result;

    // Build non secure application image ID
    image_id = IMAGE_ID(PSA_FWU_SLOT_ID_DL, FWU_IMAGE_TYPE_NONSECURE);

    printf("Active slot state before download : %s \r\n", psa_image_state[GetImageState(PSA_FWU_SLOT_ID_ACTIVE)]);
    printf("Download slot state before download : %s \r\n", psa_image_state[GetImageState(PSA_FWU_SLOT_ID_DL)]);

    // Manage fw download through XMCSS
    result=DownloadNewFirmware(image_id);

    if(!result)
    {
        return;
    }

    printf("Active slot state after download : %s \r\n", psa_image_state[GetImageState(PSA_FWU_SLOT_ID_ACTIVE)]);
    printf("Download slot state after download : %s \r\n", psa_image_state[GetImageState(PSA_FWU_SLOT_ID_DL)]);

    printf("Request firmware installation\r\n");

    result=RequestFirmwareInstall(image_id);

    if(!result)
    {
        return;
    }

    printf("Active slot state after install request : %s \r\n", psa_image_state[GetImageState(PSA_FWU_SLOT_ID_ACTIVE)]);
    printf("Download slot state after install request : %s \r\n", psa_image_state[GetImageState(PSA_FWU_SLOT_ID_DL)]);

    /* Check the image state. */
    if (GetImageState(PSA_FWU_SLOT_ID_DL) != (uint8_t)PSA_IMAGE_REBOOT_NEEDED)
    {
        printf("\rWrong Image state\r\n");
    }

    printf("Request reboot\r\n");
    res=psa_fwu_request_reboot();
    if (res != PSA_SUCCESS)
    {
        printf("Error psa_fwu_request_reboot : %ld\r\n", res);
        return;
    }
}
```



Implementation details: update_fw.c

```
HAL_StatusTypeDef Ymodem_Packet_Receive(uint8_t *pData, uint32_t uOffset, uint32_t uSize, psa_image_id_t imageId)
{
    psa_status_t status;
    /* Write the data block with block_offset 0 */
    status = psa_fwu_write(imageId, uOffset, pData, uSize);
    if (status != PSA_SUCCESS)
    {
        (void)printf("psa_fwu_write() failure");
        return HAL_ERROR;
    }
    return HAL_OK;
}
```

```
static uint32_t RequestFirmwareInstall(psa_image_id_t imageId)
```

```
{
    uint32_t ret=0;
    psa_image_id_t dependency_uuid = {0};
    psa_image_version_t dependency_version = {0};

    psa_status_t status = psa_fwu_install(imageId, &dependency_uuid, &dependency_version);

    if (status == PSA_SUCCESS_REBOOT)
    {
        printf("Install successful\r\n");
        ret = 1;
    }
    else
    {
        printf("Error install : %ld \r\n", status);
    }

    return ret;
}
```



Implementation details: update_fw.c

```
void CheckUpdateRequest(void)
{
    uint8_t cmd;
    static uint32_t FirstCall=1;
    if (FirstCall)
    {
        FirstCall=0;
        printf("Active slot state after boot : %s \r\n", psa_image_state[GetImageState(PSA_FWU_SLOT_ID_ACTIVE)]);
        printf("Download slot state after boot : %s \r\n", psa_image_state[GetImageState(PSA_FWU_SLOT_ID_DL)]);
        if (GetImageState(PSA_FWU_SLOT_ID_ACTIVE) == (uint8_t)PSA_IMAGE_PENDING_INSTALL)
        {
            psa_image_id_t image_id;
            psa_status_t res;

            image_id = IMAGE_ID(PSA_FWU_SLOT_ID_ACTIVE, FWU_IMAGE_TYPE_NONSECURE);
            res=psa_fwu_accept(image_id);
            if (res != PSA_SUCCESS)
            {
                printf("Error psa_fwu_accept : %ld\r\n", res);
            } else {
                printf("Active slot validated : %s \r\n", psa_image_state[GetImageState(PSA_FWU_SLOT_ID_ACTIVE)]);
            }
            printf("\r*****\r\n");
        }
        printf("Please press key '1' to trigger a firmware update\r\n");
    }
}
```

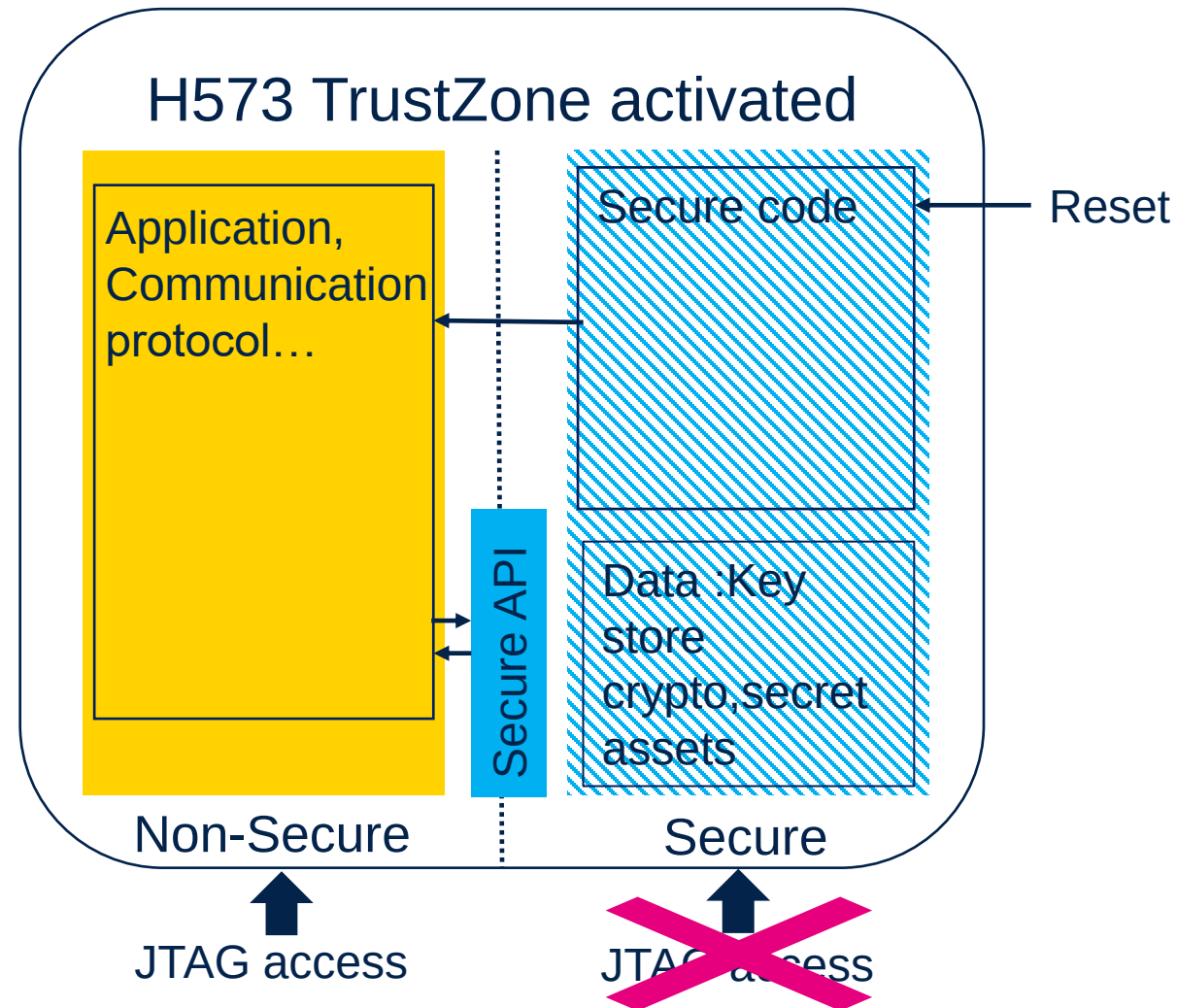


Everything is in place ?

- Our secure TimeStamped_Event_Detection has encryption of data's, have firmware update capability but....
- What is my product state ?
TZ-CLOSED
- Consequence :
 - JTAG is open when non-secure code is executed
 - The integrity and the authenticity of my application is not check at each boot (this allows to download and debug via SWD/JTAG)
- Last step : let's close the device !

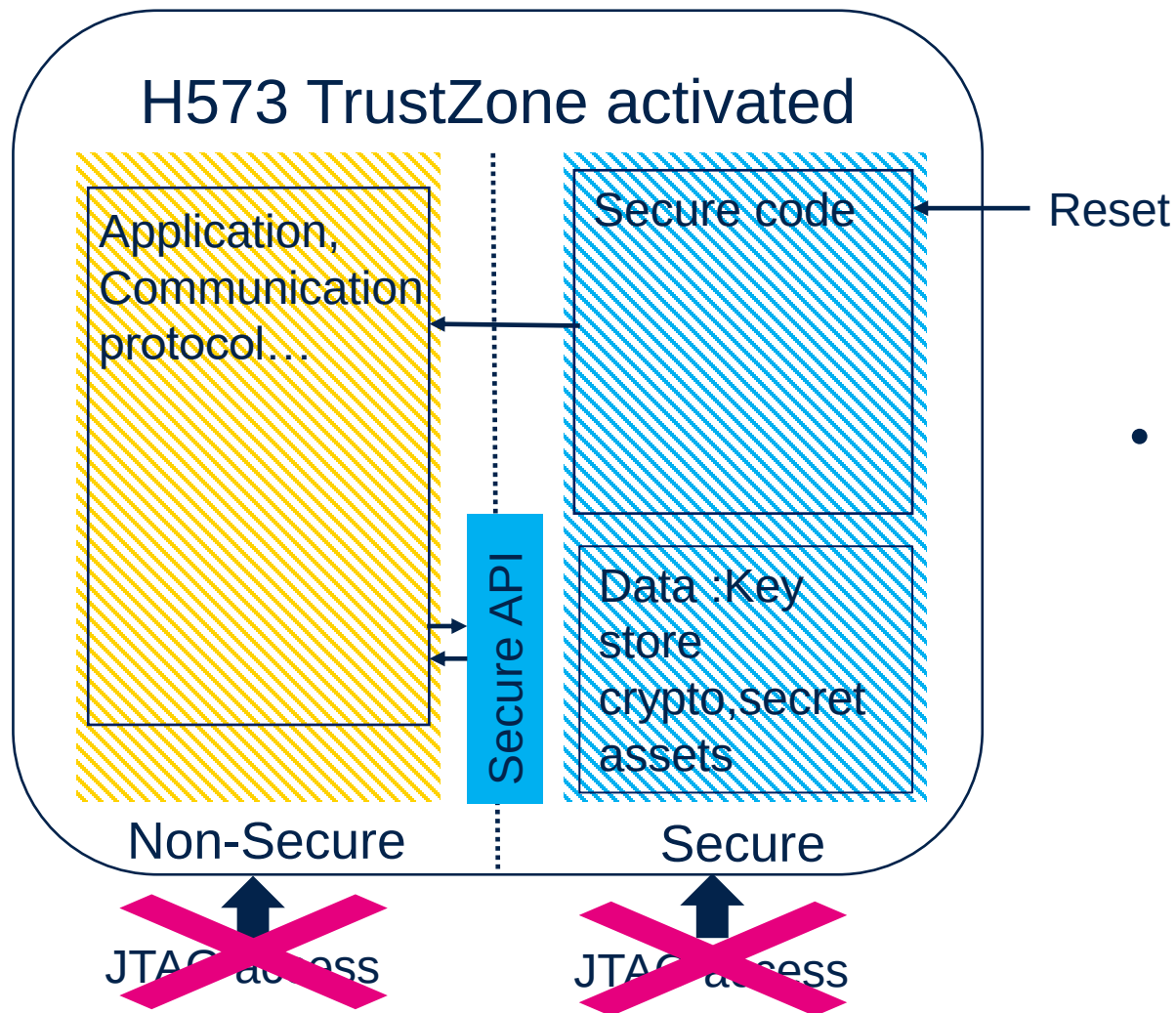


Product state TZ-CLOSED (equivalent RDP 0.5)



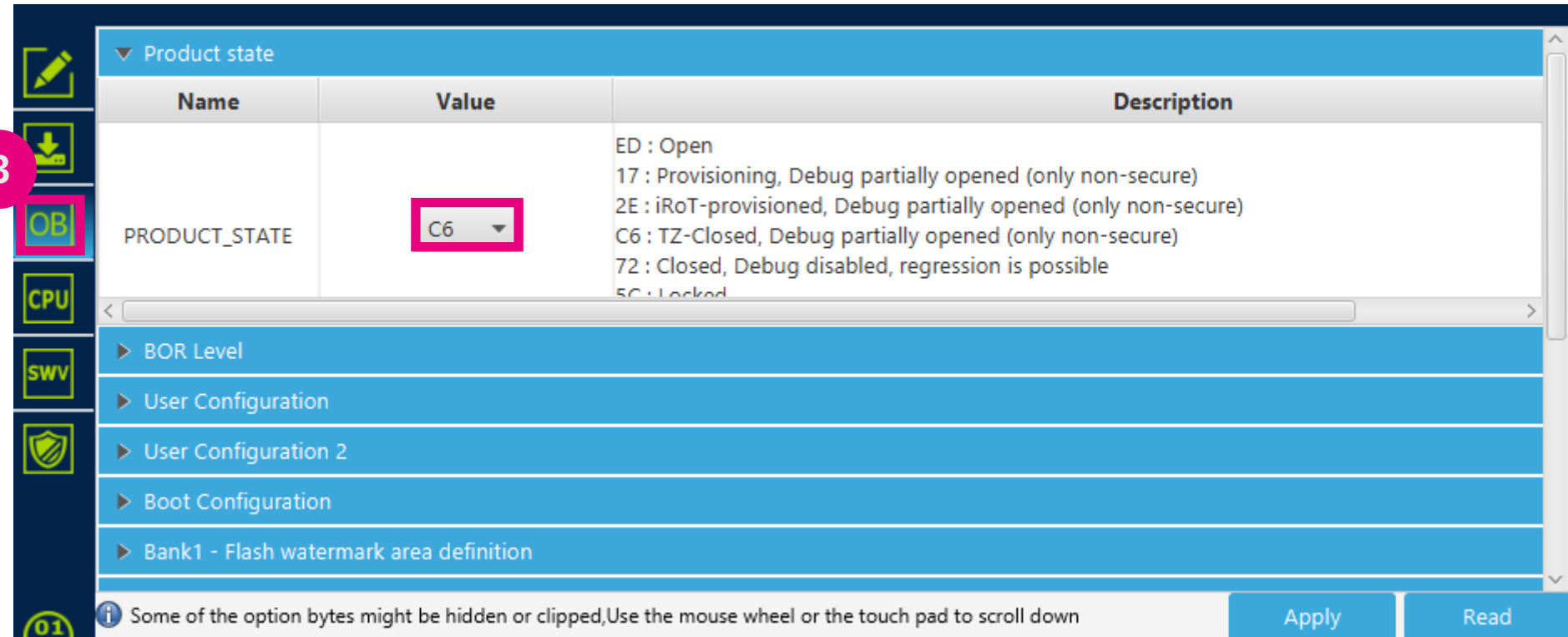
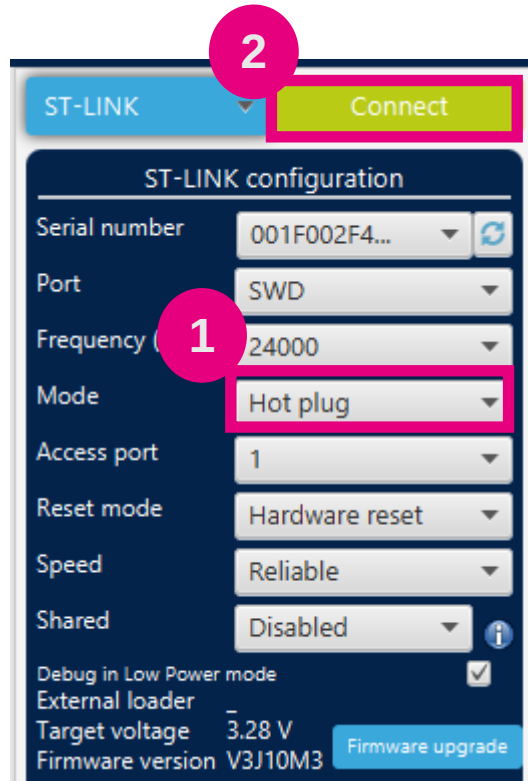


Product state CLOSED (equivalent RDP 1)

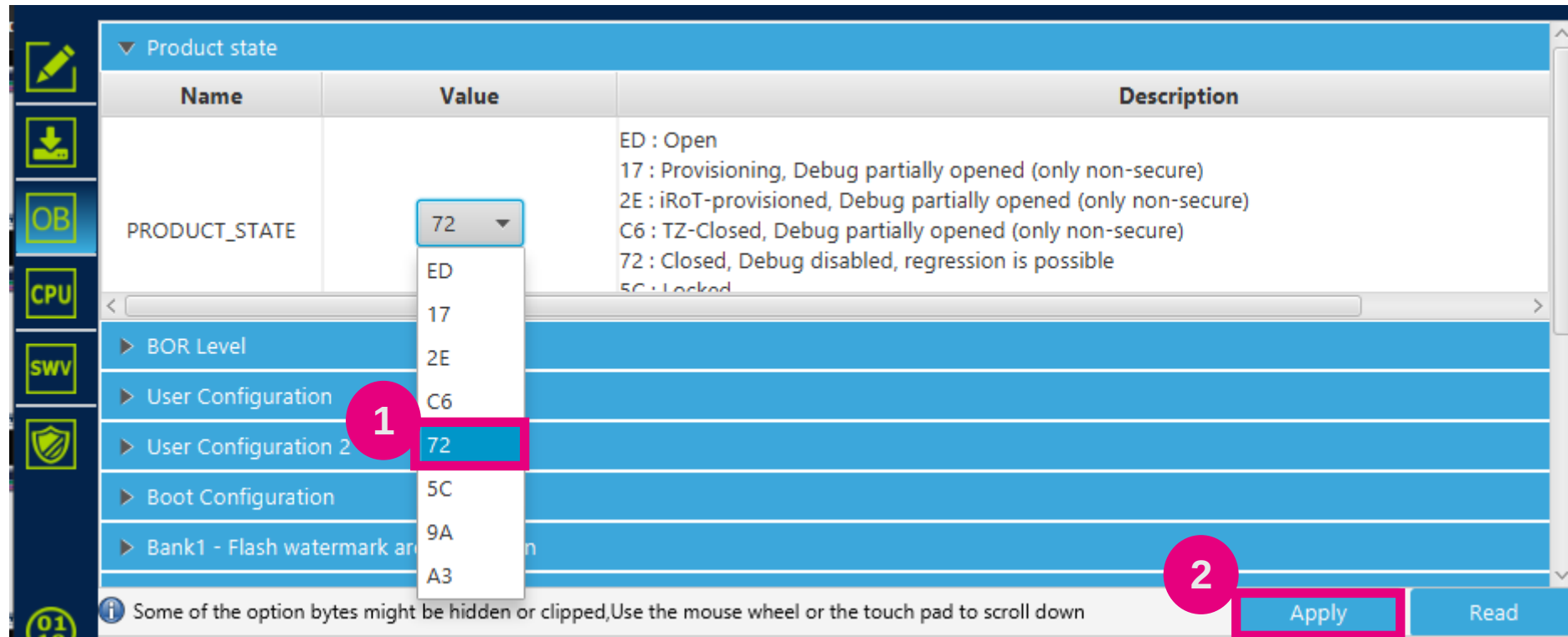


- Possibility come back to OPEN state associated with a fullchip erase

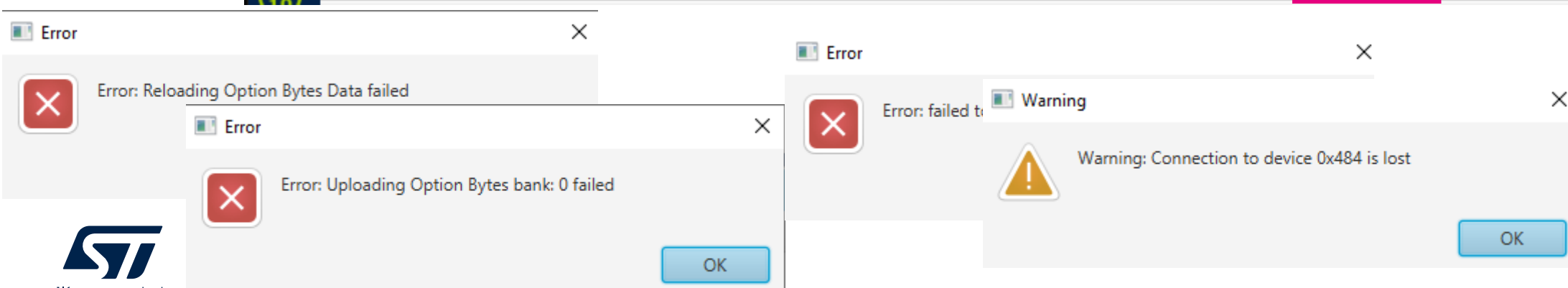
Let's close the device thanks to STM32CubeProgrammer



Let's close the device



Name	Value	Description
PRODUCT_STATE	72	ED : Open 17 : Provisioning, Debug partially opened (only non-secure) 2E : iRoT-provisioned, Debug partially opened (only non-secure) C6 : TZ-Closed, Debug partially opened (only non-secure) 72 : Closed, Debug disabled, regression is possible 5C : Locked
Some of the option bytes might be hidden or clipped, Use the mouse wheel or the touch pad to scroll down		

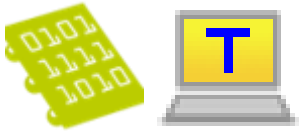


Error: Reloading Option Bytes Data failed

Error: Uploading Option Bytes bank: 0 failed

Error: failed to...

Warning: Connection to device 0x484 is lost



Application authenticity/integrity is checked at each boot !

```
COM77 - Tera Term VT
File Edit Setup Control Window Help
*****
Timestamped event detection version 3
*****
Active slot state after boot : PSA_IMAGE_INSTALLED
Download slot state after boot : PSA_IMAGE_UNDEFINED
Please press key '1' to trigger a firmware update

*****Firmware update triggered *****
Active slot state before download : PSA_IMAGE_INSTALLED
Download slot state before download : PSA_IMAGE_UNDEFINED
Please send binary application using File> Transfer> YMODEM> Send
.e_result = 0 , 3

Firmware download successful
Received bytes: 52210

Active slot state after download : PSA_IMAGE_INSTALLED
Download slot state after download : PSA_IMAGE_CANDIDATE
Request firmware installation
Install successful
Active slot state after install request : PSA_IMAGE_INSTALLED
Download slot state after install request : PSA_IMAGE_REBOOT_NEEDED
Request reboot ...

*****
Timestamped event detection version 3
*****
Active slot state after boot : PSA_IMAGE_PENDING_INSTALL
Download slot state after boot : PSA_IMAGE_PENDING_INSTALL
Active slot validated : PSA_IMAGE_INSTALLED

*****
Please press key '1' to trigger a firmware update

*****Event detected (enc) *****
cadd99ec135d8bef522385fdac597fb5a469ade87283b252d1a7f4a3cc03eab14b55584843d59e4daa09775ad4127364cf dcd1fc770ff64928fb04e9
d658839449e70d0c1ccaf3baca36c94dae40a2c31
```

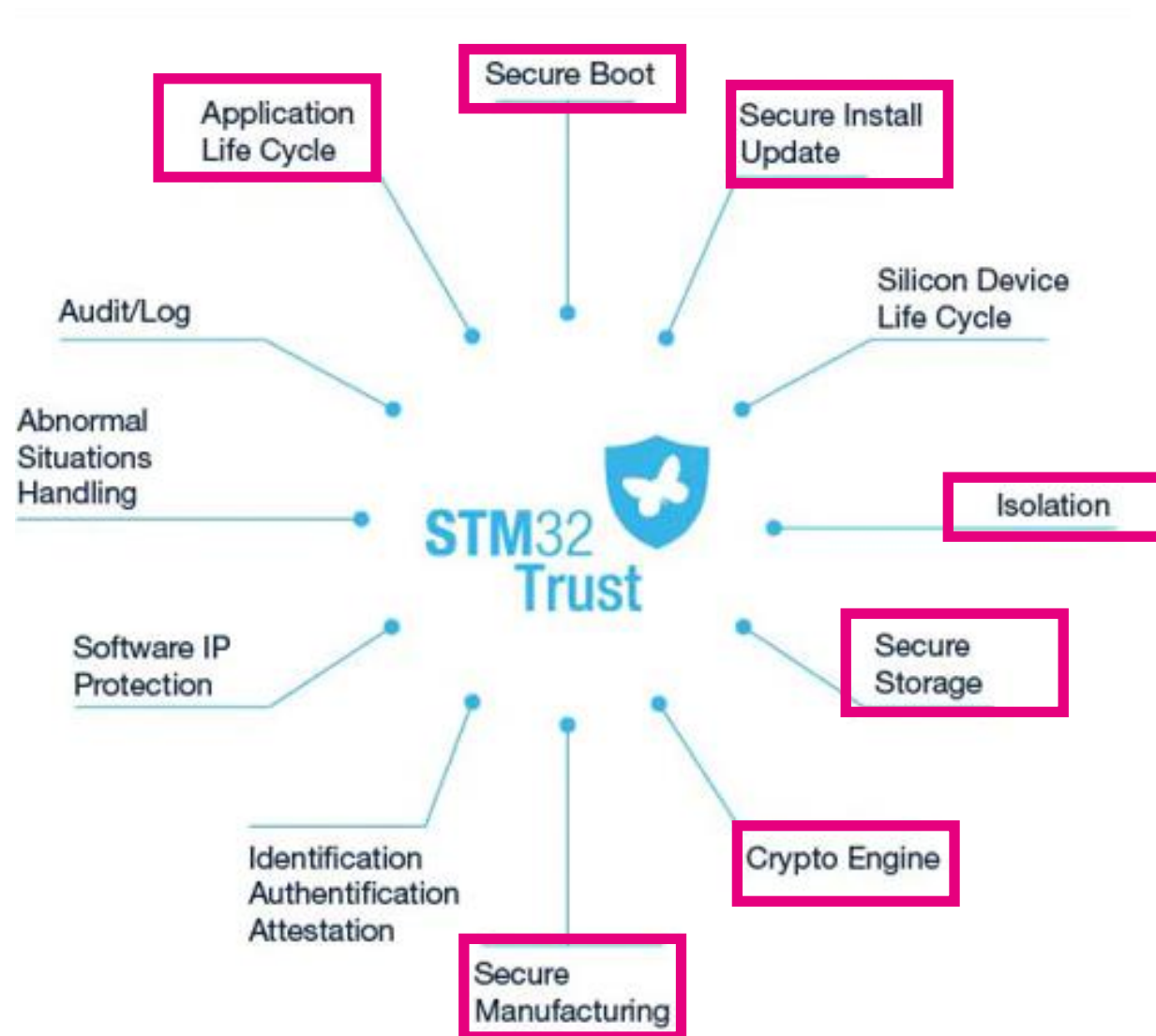


Conclusion

- PSA API somehow quite simple to integrate and to understand
- Secure boot and secure firmware update made accessible to anyone !



What about our 12 securities function...



Conclusion on Secure Manager

- Fully certified secure solution integrated in the STM32 ecosystem
- PSA API allowing access to secure services
- Pre-provisioned secure storage
- Secure update fully managed once update file is downloaded

Secure Manager features going further

- Provides 2 certificates signed with ST key allowing IOT authentication
 - 2 Cloud packages available for AWS and Azure
- Secure modules
 - Dedicated isolated secure applications
 - Either extend secure manager capabilities
 - Or 3rd party module
 - Development with specific SMDK
 - Update managed individually by STuROT
 - Secure Manager configuration to allocate more or less RAM to the modules

STM32H5 secure manager resources

https://www.st.com/content/st_com/en/wiki/wiki-portal.html

- https://wiki.st.com/stm32mcu/wiki/Security:Secure_Manager
- https://wiki.st.com/stm32mcu/wiki/Security:Secure_Manager_for_STM32H5
- https://wiki.st.com/stm32mcu/wiki/Security:SMAK_for_STM32H5
- https://wiki.st.com/stm32mcu/wiki/Security:SMDK_for_STM32H5
- https://wiki.st.com/stm32mcu/wiki/Category:How_to_start_with_Secure_Manager_on_STM32H5

Our technology starts with You



Find out more at www.st.com

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented